Problem Solving

Complete Roadmap for Junior Engineers

16	10	150+	8
Weeks Total	Topic Clusters	Practice Problems	Core Patterns

"The goal is not to memorize 150 solutions. The goal is to recognize 8 patterns that unlock them all."

How to Use This Roadmap

This document is your complete problem-solving curriculum as a junior engineer. It is structured into 10 topic clusters organized across 16 weeks. Each cluster tells you the concept, the pattern behind it, exactly which problems to solve, and in what order.

The Mindset Shift That Changes Everything

Most people practice wrong: they look at a problem, get stuck after 5 minutes, read the solution, and feel like they 'learned' it. They didn't. Real learning happens when you struggle, fail, think about WHY you failed, then rebuild the solution yourself from scratch. Every topic in this roadmap must be practiced with this loop: Attempt → Struggle → Understand the WHY → Rebuild → Revisit in 3 days.

The Practice Loop (Use for EVERY Problem)

Step	Action	Time to Spend	What You Get
1	Read the problem carefully. Write the input/output in your own words.	5 min	Prevents misunderstanding
2	Write 2-3 test cases by hand including edge cases (empty, null, single element, duplicates, negatives).	5 min	Catches 40% of bugs before coding
3	Think of the brute force solution first. Say it out loud or write it in comments.	5-10 min	Baseline to improve from
4	Ask: what is slow? Can I trade space for time? Does sorting help? Does a hashmap help?	10-15 min	Pattern recognition muscle
5	Write clean, readable code with meaningful variable names. No shortcuts.	15-20 min	Professional coding habits
6	Test your code manually with your test cases from Step 2.	5 min	Self-validation = senior trait
7	After solving: close everything, wait 3 days, solve again from scratch.	3 days later	Long-term retention

1 Big-O Complexity — The Language of Interviews

Before solving any problem, you must be able to analyze and communicate its time and space complexity. Interviewers ask this for EVERY solution you write. You cannot skip this topic.

Time Complexity — Quick Reference

Complexity	Name	What It Means	Example
O(1)	Constant	Does not grow with input size — always same speed	*Array index access, HashMap.get()*
O(log n)	Logarithmic	Halves the problem each step — very efficient	*Binary Search on sorted array*
O(n)	Linear	Visits each element once — scales with input	*Single for-loop, Linear Search*
O(n log n)	Linearithmic	Best possible for comparison-based sorting	*Merge Sort, Arrays.Sort()*
O(n²)	Quadratic	Nested loops — avoid for large inputs	*Bubble Sort, brute-force TwoSum*
O(2ⁿ)	Exponential	Doubles each step — only OK for tiny inputs	*Naive recursive Fibonacci*
O(n!)	Factorial	All permutations — unusably slow past n=12	*Brute-force permutations*

Interview Rule: Always Volunteer Complexity

Never wait for the interviewer to ask. After writing any solution, immediately say: 'The time complexity is O(n) because we visit each element once. The space complexity is O(n) for the HashMap.' This habit alone separates junior from mid-level candidates.

2 Arrays & Strings — The Foundation (Weeks 1-3)

Arrays and strings are the most common interview topic. Every other topic builds on this. Do not rush past this section even if you think you know it — the patterns here appear everywhere.

2A — HashMap / Frequency Count (Week 1)

The single most useful technique in interview problem solving. Trades O(n) space for O(n) → O(1) lookup time. Once you see this pattern, you'll use it everywhere.

The Pattern

Loop once, store values in a HashMap. Then use O(1) lookups to answer queries. Ask yourself: 'Am I looking for a complement, a pair, a count, or a duplicate?' If yes — think HashMap first.

Problem	Difficulty	Pattern Used	Key Insight
Two Sum	Easy	HashMap: value → index	Store each number's index. For each new number, check if (target - num) is already in map.
Contains Duplicate	Easy	HashSet membership	Add to set. If already present → duplicate found.
Valid Anagram	Easy	Frequency count	Count characters of s, subtract for t. All counts must reach 0.
Group Anagrams	Medium	HashMap: sorted_word → [words]	Sort each word as a key. Words with same sorted key are anagrams.
Top K Frequent Elements	Medium	HashMap + sort or Heap	Count frequencies, then extract top K using sort or min-heap.
Longest Consecutive Sequence	Medium	HashSet for O(1) lookup	Only start counting from sequence start (num-1 not in set). Avoids nested loops.
Subarray Sum Equals K	Medium	HashMap: prefix_sum → count	Key insight: if prefixSum[j] - prefixSum[i] = k, then sum of subarray i..j = k.
Two Sum II (sorted array)	Easy	Two Pointers (preview)	Use sorted property — but HashMap works too. Compare both approaches.

2B — Two Pointers (Week 1-2)

Two pointers means maintaining two index variables that move toward each other or in the same direction. Works best on sorted arrays or when you need to find pairs/triplets.

The Pattern

Start left=0, right=n-1. Move them toward each other based on a condition. When to use: 'find a pair that...', 'check if palindrome', 'remove elements in-place', 'merge sorted arrays'.

Problem	Difficulty	Pointer Movement	Key Insight
Valid Palindrome	Easy	left→, ←right, meet in middle	Skip non-alphanumeric. Compare chars from both ends.
Two Sum II (Sorted)	Easy	Move left if sum too small, right if too large	Sorted array lets us make decisions about which pointer to move.
Three Sum	Medium	Fix one, two-pointer on rest	Sort first. Fix nums[i], run two pointers on i+1..end. Skip duplicates carefully.
Container With Most Water	Medium	Move the shorter side inward	Moving the taller side can only decrease or keep width AND height — never helps.
Trapping Rain Water	Hard	Track max from left and right	Water at each position = min(maxLeft, maxRight) - height[i].
Remove Duplicates from Sorted Array	Easy	slow and fast pointers	Slow pointer marks next write position. Fast pointer scans ahead.
Squares of Sorted Array	Easy	Both ends, fill from back	Largest square is always at one of the ends due to sorting.

2C — Sliding Window (Week 2)

Sliding window is for problems about contiguous subarrays or substrings that satisfy a condition. It avoids the $O(n^2)$ nested loop by expanding and shrinking a window efficiently.

The Pattern

Maintain a window [left, right]. Expand right to include new elements. Shrink left when the window violates the constraint. The window slides forward — never resets from 0. When to use: 'longest/shortest subarray/substring with constraint X'.

Problem	Difficulty	Window Type	Key Insight
Best Time to Buy and Sell Stock	Easy	Track min so far (degenerate window)	Track min price seen. Max profit = current price - min price.
Longest Substring Without Repeating	Medium	Variable — shrink on duplicate	Use HashSet. When char already in set, shrink from left until removed.
Minimum Window Substring	Hard	Variable — shrink to minimize	Expand right until all chars covered. Then shrink left as much as possible.
Permutation in String	Medium	Fixed size = len(p)	Sliding window of fixed size. Compare frequency maps of window and pattern.
Maximum Average Subarray I	Easy	Fixed size = k	Sum window, then slide: add right element, subtract left element.
Longest Repeating Character Replacement	Medium	Variable — track max freq char	Window valid if (window_size - max_char_freq) <= k replacements.
Fruit Into Baskets	Medium	Variable — at most 2 distinct	Classic 'at most K distinct elements' sliding window.

2D — Prefix Sum (Week 3)

Prefix sum lets you answer range sum queries in O(1) after O(n) preprocessing. It is the foundation for several harder problems.

The Pattern

Build prefix[i] = sum of arr[0..i]. Then sum(i,j) = prefix[j] - prefix[i-1] in O(1). When to use: multiple range sum queries, subarray sum = target, 2D matrix sums.

Problem	Difficulty	Variation	Key Insight
Range Sum Query (Immutable)	Easy	Basic prefix sum	Precompute once, answer in O(1).
Subarray Sum Equals K	Medium	Prefix sum + HashMap	Count subarrays where prefixSum[j]-prefixSum[i]=k. Store prefix sums in map.
Product of Array Except Self	Medium	Prefix product + suffix product	No division allowed. Left product pass then right product pass.

Problem	Difficulty	Variation	Key Insight
Find Pivot Index	Easy	Left sum = total - left sum - nums[i]	At each index check if leftSum equals rightSum.
Number of Ways to Split Array	Medium	Prefix sum comparison	For each split point, check if left sum >= right sum using prefix sums.

3 Linked Lists (Weeks 3-4)

Linked list problems test pointer manipulation and the ability to reason about references. The key patterns here are fast/slow pointers and reversals — these same patterns appear in harder tree and graph problems.

Core Insight for ALL Linked List Problems

Always draw the list on paper before coding. Pointer bugs are hard to catch in your head. Draw nodes and arrows. Trace your pointer movements step by step. A 2-minute drawing saves 20 minutes of debugging.

Problem	Difficulty	Pattern	Key Insight
Reverse a Linked List	Easy	Iterative or Recursive	Store next, point current to prev, advance. Three-variable dance: prev, curr, next.
Merge Two Sorted Lists	Easy	Two pointer merge	Like merging two sorted arrays but with pointers. Use dummy head node.
Linked List Cycle	Easy	Fast/Slow pointers	Floyd's cycle detection. Fast moves 2, slow moves 1. If they meet → cycle exists.
Find Cycle Start	Medium	Fast/Slow + math	After meeting point, reset one pointer to head. Both advance by 1. They meet at cycle start.
Middle of Linked List	Easy	Fast/Slow pointers	Fast moves 2, slow moves 1. When fast reaches end, slow is at middle.
Merge K Sorted Lists	Hard	Min Heap / Divide & Conquer	Push all heads into min heap. Extract min, push its next. Repeat.
Reorder List	Medium	Find middle + Reverse + Merge	Three steps: find middle, reverse second half, merge alternating.

Problem	Difficulty	Pattern	Key Insight
Remove Nth Node From End	Medium	Two pointers with gap	Advance first pointer n steps, then move both until first hits end.
LRU Cache	Medium	HashMap + Doubly Linked List	Map gives O(1) lookup. DLL gives O(1) move-to-front and remove-from-end.
Copy List with Random Pointer	Medium	HashMap: old node → new node	Map each original node to its copy. Then set next and random pointers.

4 Stacks & Queues (Weeks 4-5)

Stacks and queues appear constantly — bracket matching, expression evaluation, BFS traversal, monotonic problems. The monotonic stack pattern in particular appears in many 'next greater/smaller element' problems.

Stack Problems

Problem	Difficulty	Pattern	Key Insight
Valid Parentheses	Easy	Stack for matching	Push open brackets. On close bracket, check if top of stack matches. Must be empty at end.
Min Stack	Medium	Two stacks or augmented stack	Keep a parallel min stack. Push current min alongside each value.
Evaluate Reverse Polish Notation	Medium	Stack for operands	Push numbers. On operator, pop two, compute, push result.
Daily Temperatures	Medium	Monotonic decreasing stack	Stack holds indices. When current temp > stack top temp → we found the 'next warmer day'.
Next Greater Element I	Easy	Monotonic stack	Build next-greater map for nums2, then look up nums1 elements.
Largest Rectangle in Histogram	Hard	Monotonic increasing stack	Stack holds indices of bars in increasing height order. Pop when smaller bar found.

Problem	Difficulty	Pattern	Key Insight
Car Fleet	Medium	Stack / sort by position	Sort by start position desc. Track time to reach target. If slower car catches up → same fleet.
Asteroid Collision	Medium	Stack simulation	Positive go right, negative go left. Collision rules: compare tops, pop as needed.

Queue / Deque Problems

Problem	Difficulty	Pattern	Key Insight
Implement Queue using Stacks	Easy	Two stacks	Input stack and output stack. Transfer when output is empty. Amortized O(1).
Sliding Window Maximum	Hard	Monotonic deque	Deque holds indices in decreasing value order. Front is always the max of current window.
Design Circular Queue	Medium	Array with head/tail pointers	Use modular arithmetic for wrapping. Track size separately.
Task Scheduler	Medium	Greedy + counting	Schedule most frequent task first. Calculate idle times mathematically.

5□ Binary Search (Week 5-6)

Binary search is not just for 'find a value in sorted array.' It is a way of thinking: eliminate half the search space at every step. Once you understand this, you can apply it to problems where the answer itself is what you're searching for.

□ The Three Templates to Know

Template 1 (classic): left<=right, return when found. Template 2 (left boundary): find first position satisfying condition. Template 3 (search on answer): binary search on the possible

answer range, not the array itself. Most medium/hard binary search problems use Template 2 or 3.

Problem	Difficulty	Template	Key Insight
Binary Search (basic)	Easy	Template 1	Classic. left=0, right=n-1, mid=(left+right)//2. Move boundaries based on comparison.
Search a 2D Matrix	Medium	Template 1 on virtual array	Treat 2D as 1D: index i maps to row=i//cols, col=i%cols.
Koko Eating Bananas	Medium	Template 3 (search on answer)	Binary search on eating speed (1 to max). Check if speed k works in h hours.
Find Minimum in Rotated Sorted Array	Medium	Template 2	The minimum is the only point where arr[mid] > arr[right]. Binary search on that property.
Search in Rotated Sorted Array	Medium	Modified Template 1	One half is always sorted. Determine which half and check if target is in it.
Time Based Key-Value Store	Medium	Binary search on timestamps	Store list of (timestamp, value). Binary search for largest timestamp <= query.
Median of Two Sorted Arrays	Hard	Partition-based binary search	Binary search on partition of smaller array. Ensure left side has (m+n+1)/2 elements.
Capacity to Ship Packages	Medium	Template 3 (search on answer)	Binary search on weight capacity. Check if all packages ship within D days at that capacity.
Find Peak Element	Medium	Template 2	If mid < mid+1, peak is to the right. If mid > mid+1, peak is at mid or left.
Split Array Largest Sum	Hard	Template 3 (search on answer)	Binary search on the maximum subarray sum. Check if we can split into at most m subarrays.

6□ Trees & Binary Search Trees (Weeks 6-9)

Trees are in almost every technical interview. The recursive nature of trees teaches you to think recursively — a skill that carries into graphs, DP, and system design. Spend extra time here.

The Tree Thinking Framework

For almost every tree problem, ask: (1) What do I need from the left subtree? (2) What do I need from the right subtree? (3) How do I combine them? (4) What is my base case (null node)? If you answer these 4 questions, you can solve 90% of tree problems recursively.

6A — Tree Traversals (Week 6)

Master all traversal types. DFS (pre/in/post order) and BFS (level order). Every other tree problem is built on one of these.

Traversal	Order	Use Case	Problems That Need It
Pre-order	Root → Left → Right	Serialize a tree, copy a tree	Serialize/Deserialize Binary Tree
In-order	Left → Root → Right	Get sorted order from BST	Validate BST, Kth Smallest in BST
Post-order	Left → Right → Root	Process children before parent	Max path sum, Delete a tree
Level-order (BFS)	Level by level	Level-based problems	Level Order Traversal, Right Side View, Zigzag

Problem	Difficulty	Traversal	Key Insight
Maximum Depth of Binary Tree	Easy	DFS post-order	1 + max(depth(left), depth(right)). Base case: null returns 0.
Invert Binary Tree	Easy	DFS pre-order	Swap left and right at each node. Recurse.
Symmetric Tree	Easy	DFS — mirror comparison	Compare left.left with right.right, left.right with right.left.
Binary Tree Level Order Traversal	Medium	BFS with queue	Use queue. Record queue size at start of each level — that's the level size.
Binary Tree Right Side View	Medium	BFS — take last of each level	Level order traversal. The last node in each level is the rightmost visible.
Diameter of Binary Tree	Medium	DFS — track global max	At each node: diameter = height(left) + height(right). Track max globally.
Path Sum II	Medium	DFS with backtracking	Pass remaining sum down. On leaf, check if sum == 0. Add/remove path on way back.

Problem	Difficulty	Traversal	Key Insight
Serialize and Deserialize Binary Tree	Hard	Pre-order DFS	Use pre-order with null markers. Deserialize reconstructs in same order.

6B — Binary Search Trees (Week 7)

BST property: left subtree values < node value < right subtree values. This property enables O(log n) search, insert, and delete on balanced trees.

Problem	Difficulty	BST Property Used	Key Insight
Validate Binary Search Tree	Medium	In-order must be strictly increasing	Track min and max bounds as you recurse. Or collect in-order and check sorted.
Kth Smallest Element in BST	Medium	In-order gives sorted sequence	In-order traversal. Decrement k at each node. Return node when k hits 0.
Lowest Common Ancestor of BST	Medium	Both values determine direction	If both < root: go left. If both > root: go right. Otherwise: root is LCA.
Insert into BST	Medium	BST search property	Recurse like a search. Insert at the null position where search ends.
Delete Node in BST	Medium	Handle 3 cases: 0, 1, 2 children	2 children: replace with in-order successor (smallest in right subtree), delete successor.
Convert Sorted Array to BST	Easy	Middle element is root	Middle of array = root. Recursively build left and right halves.
Recover BST (Two nodes swapped)	Hard	In-order finds anomaly	Find the two nodes where in-order is violated. Swap their values.

6C — Advanced Tree Problems (Weeks 8-9)

Problem	Difficulty	Approach	Key Insight
Binary Tree Maximum Path Sum	Hard	DFS — return max one-side gain	Global max tracks best path. Return to parent: node.val + max(left,right,0). Can't use both sides.

Problem	Difficulty	Approach	Key Insight
Construct Binary Tree from Preorder+Inorder	Medium	Recursion with index mapping	Preorder[0] is always root. Find root in inorder to split left/right subtrees.
Flatten Binary Tree to Linked List	Medium	Morris traversal or post-order	Post-order: flatten left, flatten right, attach right to end of left.
Populating Next Right Pointers	Medium	BFS level order	At each node: node.left.next = node.right. node.right.next = node.next.left (if exists).
Count Complete Tree Nodes	Medium	Binary search on tree height	Use BST-like binary search. Count in $O(\log^2 n)$ instead of O(n).
All Nodes Distance K in Binary Tree	Medium	Convert tree to graph	Add parent pointers (BFS), then BFS from target node up to distance K.

7 Graphs (Weeks 9-11)

Graphs are the most general data structure — trees, linked lists, and even arrays can be modeled as graphs. The main challenge is choosing the right traversal and handling visited nodes correctly.

Graph Representations to Know

Adjacency List: Dictionary/HashMap where key=node, value=list of neighbors. Most common in coding interviews. Adjacency Matrix: 2D array where matrix[i][j]=1 means edge from i to j. Good for dense graphs. Implicit Graph: Grid problems where neighbors are up/down/left/right cells — no explicit graph object needed.

7A — DFS & BFS on Graphs (Week 9-10)

Problem	Difficulty	Algorithm	Key Insight
Number of Islands	Medium	DFS/BFS flood fill	Visit each '1' cell, mark it '0', recursively mark all connected '1' cells.
Clone Graph	Medium	DFS with visited map	Map original → cloned node. For each neighbor: if not cloned yet, recurse.
Max Area of Island	Medium	DFS — return count	Flood fill, but return 1 + sum of neighbors instead of just marking visited.

Problem	Difficulty	Algorithm	Key Insight
Pacific Atlantic Water Flow	Medium	Reverse BFS from both oceans	BFS from all Pacific-border cells, BFS from all Atlantic-border cells. Intersection = answer.
Surrounded Regions	Medium	BFS from border 'O' cells	Mark all 'O' cells reachable from border as safe. Convert remaining 'O' to 'X'.
Rotting Oranges	Medium	Multi-source BFS	Add all rotten oranges to queue at once. BFS level = 1 minute. Track fresh count.
Walls and Gates	Medium	Multi-source BFS from gates	BFS from all gates simultaneously. First time a cell is visited = shortest distance to a gate.
Course Schedule	Medium	Cycle detection with DFS/BFS (Topological)	Build graph. If cycle exists → impossible. Use color states: white/gray/black.
Course Schedule II	Medium	Topological Sort (Kahn's algorithm)	In-degree array. BFS from nodes with in-degree 0. Process in topological order.
Redundant Connection	Medium	Union-Find	Add edges one by one. If two nodes already in same component → this edge creates cycle.

7B — Shortest Path & Advanced (Week 10-11)

Problem	Difficulty	Algorithm	Key Insight
Word Ladder	Hard	BFS shortest path	Each word = node. Edge exists if words differ by 1 letter. BFS finds shortest path.
Network Delay Time	Medium	Dijkstra's Algorithm	Weighted graph shortest path. Use min-heap (distance, node). Relax edges.
Cheapest Flights Within K Stops	Medium	Bellman-Ford (K iterations)	Run K+1 rounds of Bellman-Ford. Only relax edges using previous round's distances.
Swim in Rising Water	Hard	Dijkstra / Binary Search + BFS	Dijkstra where cost = max elevation on path. Min-heap on elevation.
Graph Valid Tree	Medium	Union-Find or DFS cycle check	N nodes, N-1 edges, no cycles, connected = tree.

Problem	Difficulty	Algorithm	Key Insight
Number of Connected Components	Medium	Union-Find or DFS	Count connected components using DFS or union-find.

8 Dynamic Programming (Weeks 11-14)

DP is the hardest topic but also the most teachable once you understand the method. The key is: stop thinking of DP as memorization and start thinking of it as 'recursive thinking + caching'. Every DP solution starts as recursion.

The 3-Step DP Method — Always Use This

Step 1 (Recursion): Write the recursive solution with clear base cases. Don't worry about efficiency yet. Step 2 (Memoization): Add a cache (dictionary/array) to store results. Every recursive call first checks the cache. Step 3 (Bottom-Up): If needed, convert to iterative DP table filling from smaller to larger subproblems. Start with Step 1 every time — jumping to Step 3 directly causes confusion.

8A — 1D Dynamic Programming (Week 11-12)

Problem	Difficulty	DP Definition	Recurrence
Climbing Stairs	Easy	dp[i] = ways to reach step i	dp[i] = dp[i-1] + dp[i-2]. Base: dp[1]=1, dp[2]=2.
House Robber	Medium	dp[i] = max money robbing up to house i	dp[i] = max(dp[i-1], dp[i-2] + nums[i]). Can't rob adjacent.
House Robber II	Medium	Same but circular array	Run House Robber on nums[0..n-2] and nums[1..n-1]. Take max.
Longest Palindromic Substring	Medium	dp[i][j] = is s[i..j] a palindrome	dp[i][j] = (s[i]==s[j]) && dp[i+1][j-1]. Or expand-around-center.
Palindromic Substrings (count)	Medium	Count all palindromic substrings	Expand around every center (2n-1 centers). Count each expansion.
Decode Ways	Medium	dp[i] = ways to decode s[0..i]	If s[i] valid: dp[i] += dp[i-1]. If s[i-1..i] valid (10-26): dp[i] += dp[i-2].

Problem	Difficulty	DP Definition	Recurrence
Coin Change	Medium	dp[i] = min coins to make amount i	dp[i] = min(dp[i - coin] + 1) for each coin. Classic unbounded knapsack.
Maximum Product Subarray	Medium	Track both max and min at each position	Negative * negative = positive. Track both local max and local min.
Word Break	Medium	dp[i] = can s[0..i] be segmented	For each i, try all j < i: if dp[j] and s[j..i] in wordDict → dp[i]=true.
Longest Increasing Subsequence	Medium	dp[i] = length of LIS ending at i	dp[i] = max(dp[j]+1) for all j<i where nums[j]<nums[i]. O(n²) basic, O(n log n) with patience sort.

8B — 2D Dynamic Programming (Week 12-13)

Problem	Difficulty	DP Definition	Recurrence
Unique Paths	Medium	dp[i][j] = paths to cell (i,j)	dp[i][j] = dp[i-1][j] + dp[i][j-1]. Can only move right or down.
Longest Common Subsequence	Medium	dp[i][j] = LCS of s1[0..i] and s2[0..j]	If s1[i]==s2[j]: dp[i][j]=dp[i-1][j-1]+1. Else: max(dp[i-1][j], dp[i][j-1]).
Best Time to Buy/Sell with Cooldown	Medium	States: held, sold, rest	held[i]=max(rest[i-1]-price,held[i-1]). sold[i]=held[i-1]+price. rest[i]=max(rest[i-1],sold[i-1]).
Coin Change II (count ways)	Medium	dp[i][j] = ways to make amount j using first i coins	dp[i][j] = dp[i-1][j] + dp[i][j-coin]. Unbounded knapsack.
Target Sum	Medium	dp[index][current_sum]	For each number: try adding or subtracting. Count paths reaching target.
Interleaving String	Medium	dp[i][j] = can s1[0..i]+s2[0..j] form s3[0..i+j]	dp[i][j] = (dp[i-1][j] && s1[i-1]==s3[i+j-1]) || (dp[i][j-1] && s2[j-1]==s3[i+j-1]).
Edit Distance	Hard	dp[i][j] = min edits to convert s1[0..i] to s2[0..j]	If chars equal: dp[i-1][j-1]. Else: 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]).

Problem	Difficulty	DP Definition	Recurrence
Burst Balloons	Hard	dp[i][j] = max coins from balloons i..j	Try each balloon as the LAST to burst in range. dp[i][j] = max over k of dp[i][k]+nums[i]*nums[k]*nums[j]+dp[k][j].

8C — Knapsack Patterns (Week 13-14)

Many DP problems are variants of the knapsack problem. Recognizing this reduces a new problem to a template you already know.

Variant	Key Property	Template Problems	Recurrence Pattern
0/1 Knapsack	Each item used at most once	Partition Equal Subset Sum, Target Sum	dp[i][w] = max(dp[i-1][w], dp[i-1][w-wi]+vi)
Unbounded Knapsack	Each item can be reused infinitely	Coin Change, Coin Change II	dp[i][w] = max(dp[i][w-wi]+vi, dp[i-1][w])
Bounded Knapsack	Each item has limited count	Mixed item problems	Use binary grouping to convert to 0/1 knapsack
2D Knapsack	Two constraints (weight + volume)	Profitable Schemes (LeetCode 879)	dp[i][j][k] = count with exactly j weight, k value

9 Heaps & Greedy Algorithms (Week 14-15)

Heap / Priority Queue

A heap gives you O(log n) insert and O(1) peek of min/max. Use it whenever you need the 'K largest', 'K smallest', or 'next best' element repeatedly.

When to Think Heap

Keywords: 'K largest', 'K smallest', 'K most frequent', 'K closest', 'median of stream', 'find next minimum repeatedly'. Any time you need repeated access to the current best element — heap is your tool.

Problem	Difficulty	Heap Type	Key Insight
Kth Largest Element in Array	Medium	Min-heap of size K	Keep min-heap of size K. Top of heap = Kth largest. O(n log k) vs O(n log n) sort.

Problem	Difficulty	Heap Type	Key Insight
K Closest Points to Origin	Medium	Max-heap of size K (or sort)	Keep K closest. When heap exceeds K, remove furthest. Distance = sqrt(x^2+y^2) but compare x^2+y^2.
Top K Frequent Elements	Medium	Min-heap of size K by frequency	Build frequency map. Use min-heap keyed on frequency. O(n log k).
Find Median from Data Stream	Hard	Two heaps: max-heap (lower) + min-heap (upper)	Keep halves balanced. Median = top of lower heap or average of both tops.
Task Scheduler	Medium	Max-heap + cooldown queue	Always execute most frequent available task. Re-add to heap after cooldown.
Merge K Sorted Lists	Hard	Min-heap of list heads	Push all heads. Extract min, push its next neighbor. O(n log k).
Reorganize String	Medium	Max-heap of char frequencies	Greedily pick most frequent char that isn't same as last placed.

Greedy Algorithms

Greedy makes the locally optimal choice at each step. It works when the local optimum leads to global optimum — and proving this is the hard part. For interviews, recognize the pattern; don't derive the proof.

Problem	Difficulty	Greedy Choice	Why It Works
Jump Game	Medium	At each position, track max reachable index	If current index > max reachable → stuck. Otherwise extend max reach.
Jump Game II	Medium	Always jump to position that reaches farthest	At each jump, pick the step that maximizes next max reach.
Gas Station	Medium	Track running surplus; if negative, reset start	If total gas >= total cost, a solution exists. Start from node after last failure.
Hand of Straights	Medium	Always start new group with smallest card	Sort cards. Use sorted map. Try to form groups starting from smallest available.
Merge Intervals	Medium	Sort by start, merge if overlapping	After sorting, if current.start <= prev.end → merge (extend prev.end).

Problem	Difficulty	Greedy Choice	Why It Works
Non-overlapping Intervals	Medium	Greedy by earliest end time	Keep interval with earliest end time. Greedy: fewer future conflicts.
Partition Labels	Medium	Track last occurrence of each char	Extend current partition's end to include last occurrence of every char seen so far.

Backtracking (Week 15-16)

Backtracking is structured brute force — try all possibilities, but prune paths that can't lead to a solution. It follows a clear template: choose, explore, unchoose. Every backtracking problem fits this mold.

The Universal Backtracking Template

function backtrack(state, choices): if state is a solution: add to results; return. for each choice in choices: make the choice (modify state). backtrack(updated state, updated choices). undo the choice (restore state). The 'undo' step is critical — without it you corrupt the state for sibling branches.

Problem	Difficulty	Pruning Strategy	Key Insight
Subsets	Medium	None needed (generate all)	At each element: branch to include or exclude it. 2^n subsets total.
Subsets II (with duplicates)	Medium	Skip duplicate at same depth level	Sort first. Skip nums[i] if nums[i]==nums[i-1] at same recursion level.
Permutations	Medium	Skip already-used elements	Track used[] boolean array. Try each unused element at each position.
Permutations II (with duplicates)	Medium	Sort + skip duplicate with same parent	Sort. Skip nums[i] if used[i-1] is false and nums[i]==nums[i-1] (same parent branch).
Combination Sum	Medium	Only recurse on current index and forward	Avoid duplicates by not going back. Can reuse elements (unbounded).
Combination Sum II	Medium	Skip duplicates at same level (like Subsets II)	Sort. At each level, skip if nums[i]==nums[i-1] (for i > start).

Problem	Difficulty	Pruning Strategy	Key Insight
Letter Combinations of Phone Number	Medium	No pruning needed	Map digits to letters. Try all letters for each digit. Standard backtracking.
Word Search	Medium	Mark cell as visited, unmark on backtrack	4-directional DFS. Mark cell '#' to avoid reuse, restore after backtrack.
N-Queens	Hard	Column and diagonal conflict check	Three sets: columns, diag1 (r-c), diag2 (r+c). Skip positions with conflicts.
Palindrome Partitioning	Medium	Only branch on palindrome prefixes	Check if current substring is palindrome before recursing. Prune non-palindromes.

16-Week Master Study Plan

This plan distributes all 10 topic clusters across 16 weeks at 2-3 hours/day. Each week has a clear focus, a milestone, and recommended problem count.

Week	Focus Topic	Problems to Solve	Milestone	Target Count
1	**HashMap + Frequency**	TwoSum, Contains Dup, Valid Anagram, Group Anagrams	*Can identify when to use HashMap vs brute force*	**8-10 problems**
2	**Two Pointers + Sliding Window**	3Sum, Container With Water, Longest Substr No Repeat	*Can write clean two-pointer and sliding window solutions*	**8-10 problems**
3	**Prefix Sum + Arrays Review**	Prefix Sum, Product Except Self, Subarray Sum=K + review week 1-2	*Review and solidify all array patterns learned*	**6-8 problems + 4 review**
4	**Linked Lists — Basics**	Reverse LL, Merge Sorted, Cycle Detection, Middle of LL	*Can manipulate pointers without bugs*	**8 problems**
5	**Linked Lists — Advanced + Stack**	LRU Cache, Reorder List, Valid Parentheses, Min Stack, Daily Temps	*Can combine data structures*	**8 problems**
6	**Binary Search — Basics**	Binary Search, 2D Matrix, Find Min in Rotated, Search Rotated	*Can write all 3 binary search templates from memory*	**8 problems**

Problem Solving Roadmap — Junior Software Engineer				16-Week Mastery Plan
Week	**Focus Topic**	**Problems to Solve**	**Milestone**	**Target Count**
7	**Binary Search on Answer + Tree Intro**	Koko Eating, Capacity Ship + Max Depth, Invert Tree, Level Order	*Can apply binary search to non-array problems*	**8 problems**
8	**Trees — Core DFS**	Diameter, Path Sum, Right Side View, Validate BST, Kth Smallest	*Can solve any DFS tree problem using the 4-question framework*	**8-10 problems**
9	**Trees — Advanced + BST**	LCA of BST, Construct from Preorder+Inorder, Max Path Sum	*Can solve hard tree problems under time pressure*	**6-8 problems**
10	**Graphs — DFS/BFS**	Number of Islands, Clone Graph, Rotting Oranges, Course Schedule	*Can represent graphs and traverse without getting lost*	**8-10 problems**
11	**Graphs — Topological + Union Find**	Course Schedule II, Redundant Connection, Word Ladder	*Can detect cycles and find shortest paths in graphs*	**6-8 problems**
12	**DP — 1D Patterns**	Climbing Stairs, House Robber, Coin Change, Word Break, LIS	*Can convert any 1D DP problem: recursion → memo → bottom-up*	**8-10 problems**
13	**DP — 2D Patterns**	Unique Paths, LCS, Edit Distance, Coin Change II	*Can set up 2D DP tables and identify state transitions*	**8 problems**
14	**Heap + Greedy**	Top K Frequent, Find Median Stream, Merge Intervals, Jump Game	*Can choose between heap and greedy and justify why*	**8 problems**
15	**Backtracking**	Subsets, Permutations, Combination Sum, N-Queens, Word Search	*Can write backtracking with correct pruning and state restoration*	**8-10 problems**
16	**Full Review + Mock Interviews**	Redo hardest problems from each topic. 3x mock interviews	*Can solve medium problems in 20-25 min with clear communication*	**10 review + 3 mocks**

Daily Schedule Template (2-3 hrs/day)

Time Block	Activity	How to Spend It	Expected Outcome
30 min	**Warm-up**	Solve 1 Easy problem you've seen before. Speed drill.	*Builds confidence and pattern fluency*
60-90 min	**New Topic Problem**	1 Medium problem. Use the 7-step framework. No time pressure.	*Deep understanding of the current week's pattern*

Time Block	Activity	How to Spend It	Expected Outcome
30 min	**Review**	Re-read yesterday's solution. Rewrite it without looking.	*Long-term retention through active recall*
15-30 min	**Complexity Analysis**	For every problem solved: analyze and write O() for time and space.	*Interview communication habit*

Pattern Recognition Cheat Sheet

When you see a problem for the first time, run through this checklist to identify which pattern applies. This is the skill that makes experienced engineers look 'fast' — they're not faster at coding, they're faster at recognition.

If the problem asks for...	Think of...	Key questions to ask yourself
Find a pair/triple that sums to target	**HashMap or Two Pointers**	*Is the array sorted? (Two Pointers). Unsorted? (HashMap with complement).*
Longest/shortest subarray or substring with a constraint	**Sliding Window**	*Is the constraint on a count or sum? Fixed or variable window size?*
Sum of a range of elements, multiple queries	**Prefix Sum**	*Will I query the same range many times? Yes → precompute prefix array.*
Find K largest / K smallest / K most frequent	**Heap (Priority Queue)**	*Do I need dynamic updates as I scan? Yes → heap. Static → just sort.*
Search in sorted array OR find first/last valid position	**Binary Search**	*Is there a monotone condition? Can I eliminate half the space?*
Tree/graph traversal, connected components, paths	**DFS or BFS**	*Do I need shortest path? (BFS). Do I need to explore all paths? (DFS).*
Detect cycle, shortest path, ordering of tasks	**Graph + BFS/DFS/Union-Find**	*Directed or undirected? Weighted or unweighted?*
Count/optimize solutions where subproblems overlap	**Dynamic Programming**	*Can I define a state? Does dp[i] depend on dp[i-1] or smaller subproblems?*
Generate all subsets, permutations, combinations	**Backtracking**	*Do I need to choose/unchoose? Can I prune invalid branches early?*

If the problem asks for...	Think of...	Key questions to ask yourself
Schedule tasks, merge intervals, maximize minimum	**Greedy**	*Does picking the local optimum always lead to global optimum?*
Nested brackets, next greater element, monotone stack	**Stack**	*Is there a 'waiting' relationship between elements? Stack holds waiting elements.*
Kth node from end, cycle start, middle of list	**Fast/Slow Pointers**	*Are two pointers moving at different speeds through the same structure?*

Resources & Platforms

Resource	What It's Best For	How to Use It
NeetCode.io	Structured roadmap organized by pattern with video explanations	Follow the NeetCode 150 list in order. Watch video ONLY after attempting yourself.
LeetCode	Largest problem bank — filter by topic and company	Use topic filters to practice current week's pattern. Track problems in a notebook.
Pramp.com	Free live mock interviews with peers	Do 1 mock interview per week starting from Week 8. Practice talking while coding.
LeetCode Discuss	See alternative approaches from the community	After solving, read top voted solutions to find more elegant approaches.
CS Dojo / NeetCode YouTube	Video explanations of patterns and problems	Watch to understand a PATTERN, not individual solutions.
Grind 75 (techinterviewhandbook.org)	Curated list of 75 essential problems	Use as a final review list in Weeks 14-16 before interviews.

Your 16-Week Commitment

Follow this roadmap and in 16 weeks you will:

✓ Recognize patterns instantly ✓ Solve mediums in 20-25 minutes

✓ Communicate complexity clearly ✓ Adapt when interviewers change the problem

You already have the mindset. Now you have the map.